LangGraph Core Concepts

→ What is LangGraph?

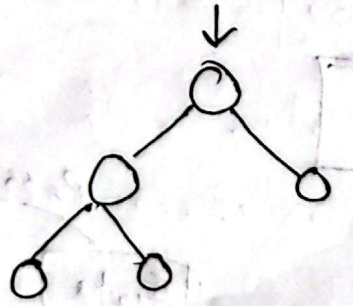
↳ LangGraph is an orchestration framework for building intelligent, stateful, and multi-step LLM workflows.

LLM workflow $\xrightarrow{\text{convert}}$ graph

↓

flowchart Representation → execution

then



→ It enables advanced features like parallesim, loops, branching, memory and resumability - making it ideal for agentic and production-grade AI applications.

LLM workflows

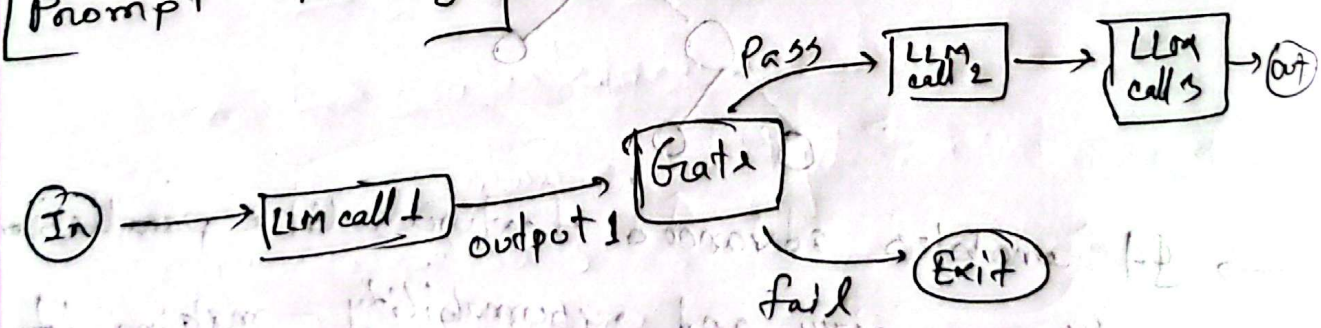
→ LLM workflows are a step by step process using which we can build complex LLM applications.

→ Each step in a workflow performs a distinct task - such as prompting, reasoning, tool calling, memory access or decision-making.

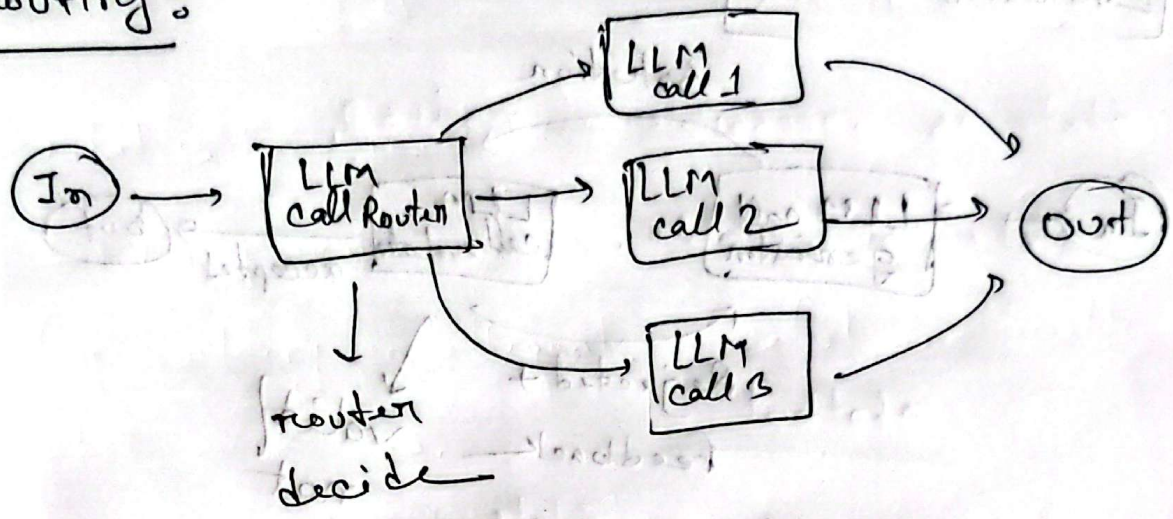
→ Workflows can be linear, parallel, branched or looped, allowing for complex behaviours like retries, multi-agent communication or tool-augmented reasoning

→ common workflows

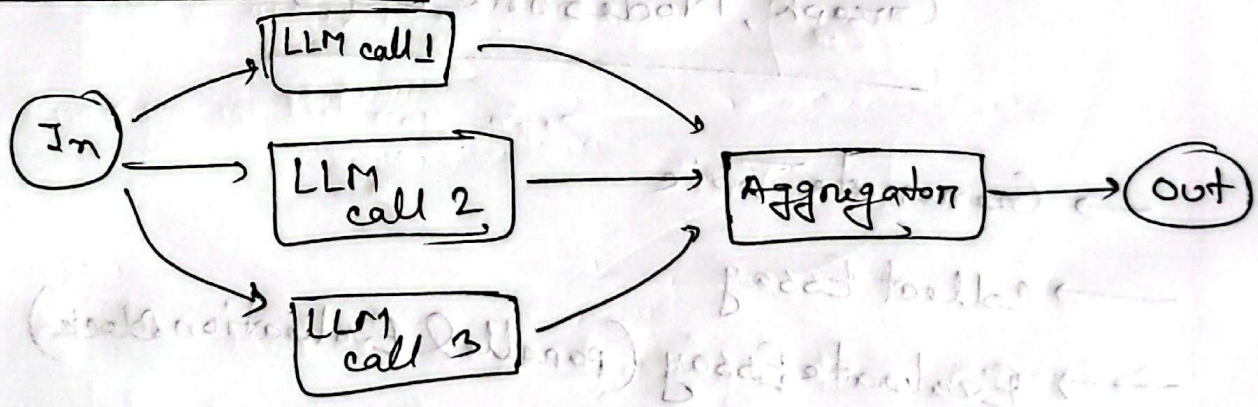
[Prompt chaining]



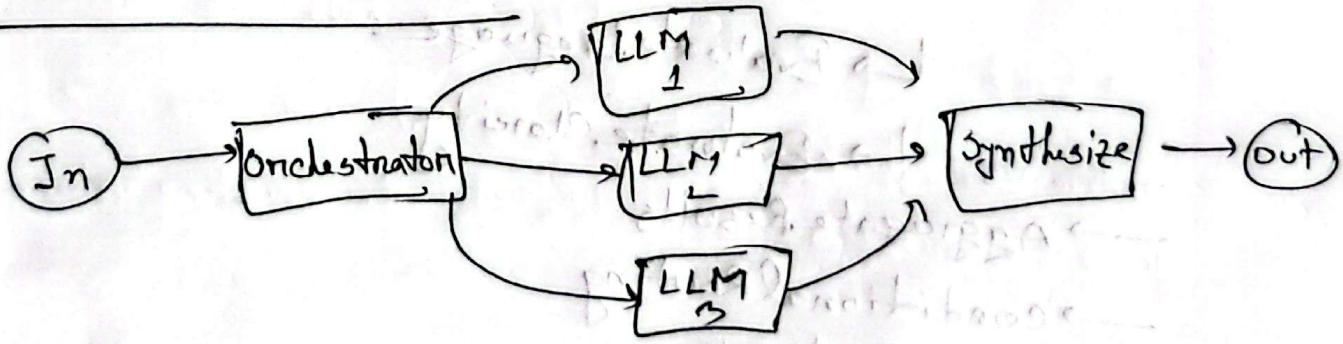
Routing:



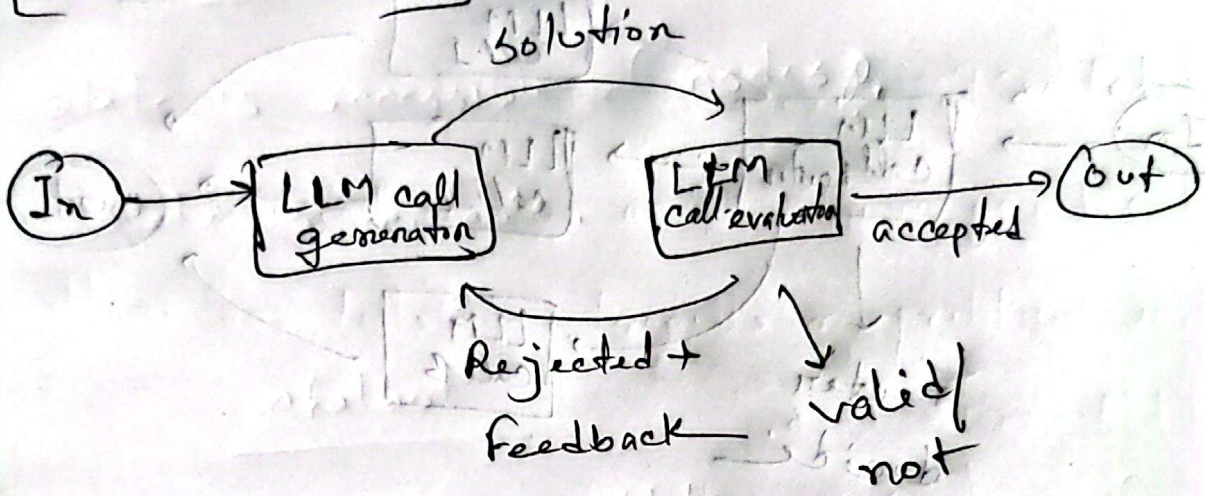
Parallelization:



Orchestrator workers

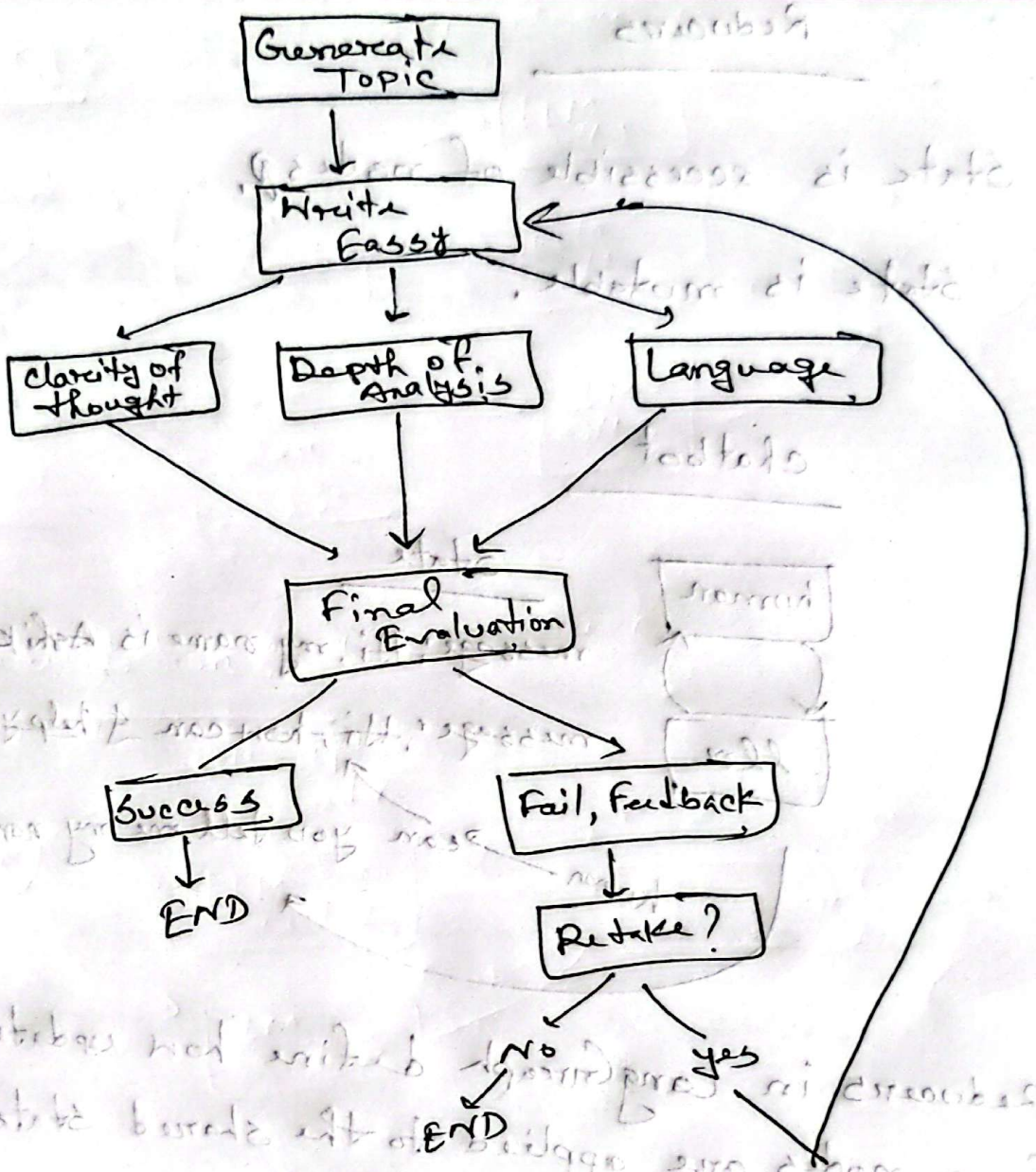


Evaluator Optimizer



Graph, Nodes and Edges

→ Generate Topic
→ Collect Essay
→ Evaluate Essay (parallel Evaluation Block)
  ↳ Evaluate Depth
  ↳ Evaluate Language
  ↳ Evaluate Clarity
→ Aggregate Results
→ Conditional Routing
→ Give Feedback
→ Collect Revision
→ Show Success



State

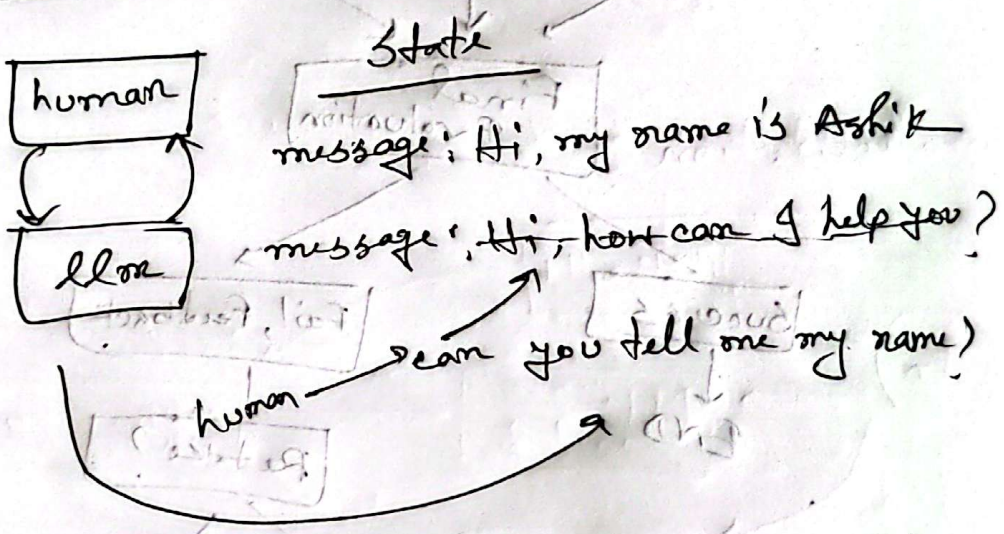
→ In LangGraph, State is the shared Memory that flows through your workflow - it holds all the data being passed between nodes as your graph runs.

Reducers

State is accessible of nodes.

State is mutable.

chatbot



Reducers in LangGraph define how updates from nodes are applied to the shared state.

→ Each key in the state can have its own reducers, which determines whether new data replaces, merges, or adds to the existing value.

LangGraph Execution Model

1. Graph Definition
   - ↳ State schema
   - ↳ Nodes
   - ↳ Edges

2. compilation
   - .compile()

3. Invocation
   - .invoke(initial_state)

4. Super-Steps Begin
   - → Execution proceeds in rounds

5. Message Passing & Node Activation

6. Halting condition

CamScanner